



Scheduling algorithms for heterogeneous batch processors with incompatible job-families

M. MATHIRAJAN^{1*}, A. I. SIVAKUMAR¹ and V. CHANDRU²

¹*Singapore-MIT Alliance, School of MPE, NTU, Singapore 639 798*

E-mail: mmathirajan@ntu.edu.sg

²*Department of Computer Science and Automation, IISc, Bangalore 560 012, India*

Received October 2002 and accepted December 2003

We consider the problem of scheduling heterogeneous batch processors (i.e., batch processors with different capacity) with incompatible job-families and non-identical job sizes to maximize the utilization of the batch processors. We analyzed the computational complexity of this problem and showed that it is NP-hard and proposed eight variants of a fast greedy heuristic. A series of computational experiments were carried out to compare the performance of the heuristics and showed that the heuristics are capable of consistently obtaining near (estimated) optimal solutions with very low-computational burden for large-scale problems. We also carried out a study to find the effect of family processing time changes on the performance of the heuristics. This sensitivity analysis indicated that the processing time set of job-families influences the performance of the heuristic algorithms.

Keywords: Heterogeneous batch processors, incompatible job-families, non-identical job-sizes, heuristics, scheduling

1. Introduction

A batch processor (BP), or a batch-processing machine (BPM), is one which processes a number of jobs simultaneously as a batch with common beginning and end times. Although batch processors are encountered in many different environments, such as diffusion operations in semiconductor industries or heat-treatment operations in metalworking industries, relatively few research works have addressed problems related to this kind of production equipment. This research study considers the problem of scheduling heterogeneous batch processors in the presence of dynamic job arrivals (i.e., the full knowledge is available about future arrival of jobs) with incompatible job-families and non-identical job sizes.

The general motivation on batch processor scheduling is the wide application of batch processors in manufacturing systems such as semiconductor manufacturing, aircraft industries, shoe factories, metal industries, etc. Particularly this research is motivated by heat-treatment furnace operations in steel casting manufacturing. Since the processing time at heat-treatment furnace operations is often longer compared to other operations in the steel-casting foundry, it affects the scheduling and the overall flow time considerably. Further, heat-treatment furnace operation is important because it determines the final properties that enable components to perform under demanding service conditions such as large mechanical load, high temperature, and anti-corrosive processing. Generally, different types of castings have to undergo more than one type of heat-treatment operation, where the total heat-treatment processing times change. For control purposes, castings are primarily classified into a number of job-families

*Corresponding author.

based on the alloy type such as low-alloy castings and high alloy castings. These families are further classified into various sub-families based on the type of heat-treatment operations required by them. For technical reasons, such as type of alloy, temperature level and the expected combination of heat-treatment operations, the castings (jobs) from different families cannot be processed together in the same batch. These job families are therefore mutually incompatible for processing.

Further, a major concern of management in general is to maximize throughput and reduce flow time and work-in-process inventories (WIP). This is motivated by the consideration of maximizing the utilization of the batch processors as the objective.

The plan of the paper is as follows. In Section 2, we review previous related work on scheduling batch processors with incompatible job families. We analyze the computational complexity of the problem described in this paper and propose eight variants of a fast greedy heuristic in Section 3. Extensive empirical analysis carried out for (a) comparing the performance of the heuristics with estimated optimal value and (b) relative effectiveness is presented in Section 4. A simple sensitivity analysis carried out on the performance of the proposed heuristic algorithm is discussed in Section 5. A summary and discussion of future research directions concludes the paper.

2. Previous related research

Recently, much research has been reported on scheduling problems concerned with BPs, assuming all problem data such as processing times and job arrival times are known and deterministic (Mathirajan and Sivakumar 2003). Instead of discussing a complete review of deterministic scheduling BPs, the closely related previous research work is discussed here.

Scheduling BPs with incompatible job families under a variety of objective functions has been addressed by a number of researchers. Dobson and Nambimadom (1994, 2001) studied the problem of minimizing weighted completion time on a BP where each job requires a different amount of machine capacity. They provided an integer programming formulation, in which they used a lower bound on the value of the optimal solution. They also proved

that this problem is NP-hard and provided a number of heuristic algorithms.

Uzsoy (1995) examined the problem of scheduling batch processors to optimize a number of performance measures such as minimizing maximum lateness, minimizing weighted completion times, etc. and provided exact as well as approximate solutions for a number of problems involving single and parallel BPs. Uzsoy (1995) also addressed the problems of minimizing the makespan and minimizing the maximum lateness on a single BP in the presence of dynamic job arrivals.

Fanti *et al.* (1996) addressed the problem of scheduling a single BP with incompatible job-families. This problem is observed in a typical shoe factory and they mentioned that this operation is a bottleneck of the whole manufacturing process of shoe manufacturing. They aimed to maximize the utilization of the BP and to minimize the makespan. Due to the computational burden of the problem in getting optimal scheduling, they proposed heuristic algorithms. They demonstrated the efficiency of the proposed heuristic algorithm using a case study data.

Zee *et al.* (1997) proposed “dynamic job assignment heuristic (DJAH)” for scheduling parallel identical BPs with incompatible job-families, observed in the aircraft industry. The objective of DJAH is to minimize the costs associated with a job and consists of linear waiting costs and a fixed amount of set up costs. From extensive computational experiments carried out, they demonstrated the efficiency and effectiveness of the proposed heuristic.

Hung (1998) studied the E-beam writer in the mask manufacturing process as scheduling a single BP and studied the same problem with parallel identical BPs situation along with two families of jobs (that is, two mask sizes), which are incompatible in nature in constructing a batch of jobs for processing. A dynamic programming (DP) formulation was proposed to schedule E-beam writer to minimize the total tardiness. Computational experiments showed that the proposed DP reduced the total tardiness by an average of 55% from the mask shop’s current method.

Kempf *et al.* (1998) studied the problem of scheduling a single BP with non-identical job sizes to minimize the total completion time and makespan subject to the primary constraint on capacity of the BP and the secondary resource requirements, viz., the availability of board. Several heuristics have been developed for this system. From the extensive

computational experiments, they have claimed that the heuristics developed are capable of producing consistently good solutions in modest CPU times.

Mehta and Uzsoy (1998) examined the problem of scheduling a BP to minimize total tardiness. They developed a DP algorithm, which is similar to that of Uzsoy *et al.* (1992). Further, they provided two heuristic algorithms for minimizing mean tardiness for large size problems. The first is the modification of the apparent tardiness cost (ATC) rule (called BATC) proposed by Vepsalainen and Morton (1987) and the second closely follows the decomposition heuristic of Chambers *et al.* (1991) for minimizing the number of weighted tardiness on a single machine. They conducted a series of computational experiments and claimed that DP provided optimal solutions in a reasonable amount of computation time for small problems and in many problem instances the heuristic BATC was expected to provide reasonably good solutions very fast.

Devpura *et al.* (2000) studied the diffusion operation as a single batch processing machine with incompatible job families, and the performance measurement as minimizing the total weighted tardiness. They looked at various heuristic approaches and introduced swapping algorithm. From the series of computational experiments carried out, they showed that swapping jobs between already sequenced batches could provide a lot of improvement on the performance of the heuristics.

Kim *et al.* (2000) considered scheduling BP in wafer fabrication with multiple product types and distinct due dates for each job type. They focussed scheduling objective of minimizing the total tardiness of the orders. They proposed a two stage heuristic approach. Simulation experiments were carried out to compare the proposed heuristics with the heuristic minimum batch size (MBS), suggested by Neuts (1967). Results of the computational experiments showed that the proposed heuristics are effective and efficient in comparison with MBS.

Azizoglu and Webster (2001) extended the analysis of the model studied by Dobson and Nambimadom (1994, 2001). The optimization algorithm proposed in Azizoglu and Webster (2000) was appropriately customized for scheduling a BP with incompatible job-families. Here the jobs in a given family were considered to have identical processing times, arbitrary job weights and arbitrary job sizes. This algorithm yielded optimal total weighted completion

time to problems up to about 25 jobs in reasonable CPU time. Further, they highlighted that as this algorithm generates very quickly a feasible schedule in the first phase this can be used as a heuristic solution for larger problems.

Zee *et al.* (2001) proposed a new heuristic: dynamic scheduling heuristic (DSH) for scheduling parallel, non-identical BP with incompatible job-families. They assume that a limited number of near future arrivals are known or predicted. This heuristic was compared with the heuristic DJAH, which was developed by them for the case of parallel, identical with incompatible job-families. From the extensive computational experiments carried out, they showed that the new heuristic DSH is consistently given an improved system performance in comparison with DJAH.

Our research differs from the above reviewed work in three ways. First, we examine parallel and non-identical batch processors, observed in the steel casting industries. Second, in our model, we consider three problem characteristics (a) incompatible job-family, (b) non-identical job size and (c) dynamic job arrival together. Last, we consider wide range of individual job sizes in compared with the capacity of the BPs.

In addition to the above, the scheduling of heat-treatment furnaces does not seem to have been addressed in the literature to date, and is of considerable interest to management owing to a large proportion of the total production time being accounted for by the processing times involved in these operations. Further, steel-casting manufacturing poses some of the most difficult scheduling problems (Shekar, 1998).

3. Scheduling algorithms

The scheduling of heterogeneous BPs with incompatible job-families and non-identical job sizes involves three interrelated sets of decision and they are (1) batch processor selection (BPS), (2) job-family selection (JFS), and (3) batch construction (BC). In this section, we provide the assumptions made in modeling the scheduling problem, complexity of the scheduling problem and the proposed heuristic scheduling algorithms.

3.1. Assumptions

- All BPs are continuously available.
- All jobs must pass through the operation(s) to be carried out at the BPs.
- The scheduling horizon is assumed to be of seven days and at the beginning time of the scheduling, number of jobs corresponding to the first-day of the planning horizon is assumed to be of available in the WIP.
- At the beginning of every fixed interval of time (in our analysis, every 24 h)¹ from starting time of scheduling, a number of jobs will arrive for operations at the BPs (i.e., a full knowledge is available about future arrival of jobs).
- The batch size for any BP is independent of the shape (i.e., independent of dimension) of a job but is dependent on the size (i.e., dependent on weight in terms of kg of a job) and the capacity (in terms of total acceptable kg and not in terms of dimension) of the BP.
- The availability of the number of hardware (trays) in which the jobs are normally placed for a specific operation at the BP is assumed to be unlimited and, thus, does not affect the scheduling decisions. Further, there are no limiting resources other than the BPs.
- Processing times of job-families are considered constant and independent of batch size.

3.2. Heuristic scheduling algorithms

Scheduling “heterogeneous BPs in the presence of known future job arrivals with incompatible job-families and non-identical job sizes to maximize the utilization of the batch processors” is NP-hard. This is because the problem configuration defined here is a superset to the problem configuration defined in Dobson and Nambimadom (2001), who proved that their problem is NP-hard. Thus, there is a good reason to look for heuristic methods that will produce solutions that are efficient and effective.

In this paper, the design issues underlying the proposed heuristic algorithms are related to managerial considerations in the scheduling of heat treatment operations in steel casting manufacturing and they are:

- The heuristics for batch processor selection (BPS) are based on the “capacity” of batch processors available.

- The heuristics for job-family selection (JFS) are based on computing a family-wise index and selecting the job-family, which has the minimum index (the index is computed using processing time of a family, number of jobs in the family, weight/size of each job and the priority² of each job in the family).
- The heuristics for batch construction (BC) are dependent on the selection of a set of feasible jobs from the top of a sorted-list of jobs. (The sorting is based on the “job priorities” in ascending order and within the priority, is based on “job weights” in descending order.)

Based on the above managerial considerations, eight variants of a greedy heuristics are proposed for the research problem. Notations used in the algorithms and the systematic procedure of the algorithms are given below:

Notation:

F = job-family F and $F = 1, 2, \dots, NF$

NF = number of job-families

$INDEX(F)$ = an $INDEX$ computed for a job-family F

$PT(F)$ = processing time of job-family F

$APJ(F)$ = average priority of jobs of the job-family F

$ASJ(F)$ = average size of jobs of the job-family F

$WAPJ(F)$ = weighted average priority of jobs of the job-family F with job-size as weight

$WASJ(F)$ = weighted average size of jobs of job-family F with job-priority as weight

$MSJ(F, BP)$ = mean job-size of job-family F based on a subset or batch of jobs consistent with the selected BP

$MPJ(F, BP)$ = mean priority of job-family F based on a subset or batch of jobs consistent with the selected BP

$WMSJ(F, BP)$ = weighted mean job-size of job-family F based on a subset or batch of jobs consistent with the selected BP; here the priority is considered as weight

$WMPJ(F, BP)$ = weighted mean priority of job-family F based on a subset or batch of jobs consistent with the selected BP; here the job-size is considered as weight

Heuristic algorithm 1 (A1)

Decision 1: Batch processor selection (BPS)

Step 1: Whenever a tie occurs in selecting a BP for

scheduling, select a BP, which has a maximum capacity one; when there is no tie situation for selecting a BP for scheduling, select the BP which is going to be available early for the next scheduling.

Decision 2: Job-family selection (JFS)

Step 2: Compute for each job-family an index, $INDEX(F) = \{PT(F)/WASJ(F)\}$.

Step 3: Select a job-family, which has the Min $\{INDEX(F), F = 1, 2, \dots, NF\}$

Decision 3: Batch construction (BC)

Step 4: Sort, all jobs of the selected job-family (i) based on job's arrival day in ascending order, (ii) within the day based on job's priority in ascending order and (iii) within the priority based on job's size in descending order.

Step 5: Select a set of feasible jobs (a job is feasible only if it is possible to accommodate into the selected BP based on its weight, that is in terms of kg) from the top of a sorted-list until the selected BP capacity (that is, its total weight in terms of kg) is utilized to the maximum extent.

Step 6: Update the Time matrix (for each BP, a time matrix is created and this matrix contains information on starting time of each batch and ending time of the corresponding batch) of BP availability and the sorted-list of the selected job-family.

Step 7: Repeat Steps 1–6 until all jobs for the given planning period are scheduled.

Note: Heuristic algorithms 2 to 4 that follow are identical to heuristic algorithm 1 except in Step 2. Therefore, only the modified Step 2 is detailed below for each of the variants:

Heuristic algorithm 2 (A2)

Step 2.4: Compute for each job-family an index, $INDEX(F) = \{PT(F)/WAPJ(F)\}$

Heuristic algorithm 3 (A3)

Step 2.4: Compute for each job-family an index, $INDEX(F) = \{PT(F)/APJ(F)\}$

Heuristic algorithm 4 (A4)

Step 2.4: Compute for each job-family an index, $INDEX(F) = \{PT(F)/ASJ(F)\}$

Heuristic algorithm 5 (A5)

The heuristic algorithm MA1 is a modified version of the heuristic algorithm A1 and therefore, only the

modified Steps of A1 is given below:

Decision 2: Job-family selection (JFS)

Step 2: For each family, temporarily construct a batch using the following steps:

Step 2.1: Sort, all the jobs of the job-family F based on job's arrival day in ascending order, and within a day based on job's priority in ascending order and all daily jobs within a priority based on job's size in descending order.

Step 2.2: Select a set of feasible jobs from the top of a sorted-list until the selected BP capacity is utilized to the maximum extent.

Step 2.3: Do steps 2.1 and 2.2 for all job-families. Further, store temporarily all the details of the jobs considered for the batch of jobs corresponding to each family for the selected BP.

Step 2.4: Compute for each job-family an index, $INDEX(F) = \{PT(F)/WMSJ(F, BP)\}$.

Step 3: Select a job-family, which has the Min $\{INDEX(F), F = 1, 2, \dots, NF\}$.

Decision 3: Batch construction (BC)

Step 4: Confirm the temporarily constructed batch corresponding to the selected job-family as per step 3 for scheduling at time t .

Step 5: Update the time matrix of batch processor availability and the sorted-list of the selected job-family.

Step 6: Repeat Steps 1–5 until all jobs for the given planning period are scheduled.

Note: Heuristic algorithms 6–8 that follow are identical to MA1 except in Step 2.4 of the algorithm. Therefore, only the modified Step 2.4 is detailed below for each of the variants:

Heuristic algorithm 6 (A6)

Step 2.4: Compute for each job-family an index, $INDEX(F) = \{PT(F)/WMPJ(F, BP)\}$

Heuristic algorithm 7 (A7)

Step 2.4: Compute for each job-family an index, $INDEX(F) = \{PT(F)/MPJ(F, BP)\}$

Heuristic algorithm 8 (A8)

Step 2.4: Compute for each job-family an index, $INDEX(F) = \{PT(F)/MSJ(F, BP)\}$

It is to be noted that, in the proposed eight variants of the greedy heuristic algorithm, the heuristics for both

BPS and BC are kept fixed, and the heuristics for JFS is only *changed* in each of the variants. Further, the variation in the heuristics for JFS is based on the four criteria (1) the weighted average size of a job, (2) the weighted average priority of a job, (3) the simple average priority of a job, and (4) the simple average size of a job. Accordingly, the proposed eight variants of a greedy heuristic are grouped into two sets. Each set has four heuristic algorithms. In the first set of four algorithms, the Index, to be used for deciding JFS, is being computed using all the jobs available in each Job-Family irrespective of the capacity of the selected BP. Whereas, in the second set of four algorithms, for computing the index, we use only a subset or batch of jobs from each job-family that is consistent with the capacity of the selected BP. All the eight variants of the greedy heuristic are implemented using Turbo C++.

4. Computational experiments

To determine how well the proposed heuristics perform, we carried out computational experiments using an experimental approach. An experimental approach relies on two elements and they are (a) experimental design and (b) a measure of effectiveness. Further, in this study, the analysis of the experimental data is presented in two perspectives. The goal of the first perspective was to evaluate the absolute quality of the solutions obtained by the proposed heuristic algorithms when compared with estimated optimal solutions. The second perspective was to compare the relative performance of the proposed heuristic algorithms both in terms of computational effort and quality of the solutions. The details of the measure of effectiveness, experimental design and the evaluation of the heuristic algorithms are given in the following sections.

4.1. Measure of effectiveness

Since the performance of the algorithms may vary over a range of problem instances, the performances of the proposed heuristic algorithms are compared using the following two standard measures:

(a) average relative percentage deviation (ARPD): For each run (problem instance) we first compute the relative deviation (RD) of the solution obtained by the

different heuristic algorithms to a key-solution. In the planned computational experiment 1 (Section 4.3), the key-solution is an estimated optimal solution and in the planned computational experiment 2 (Section 4.4), the key-solution is the best solution obtained for a given problem instance from any of the proposed heuristic algorithms. Further, we compute the (ARPD) of the solution obtained by the different heuristics over the number of runs (problem instances). This measure provides the average performance of the heuristics.

(b) maximum relative percentage deviation (MRPD): Using the RD obtained for each run, the MRPD will be obtained over the number of runs (problem instances). This MRPD measures the worst-case performance of the heuristics.

4.2. Experimental design

The experimental design is the process of planning an experiment to ensure that the appropriate data will be collected/generated. Based on the observation made in the steel casting industry, we identified three important input parameters, which could affect the performance of the heuristic algorithms and they are number of jobs (n), job-priority (P) and job-family (F). In the observed steel-casting foundry, each job is assigned a priority. In the experimental design, P varies from 1 to 8, which is based on the observed steel casting industry. Further, most of the time, in the observed steel-casting foundry, the jobs are classified into five different groups (F) based on the expected combination of heat treatment operations required by the job.

In addition to the above input parameters required for our scheduling problem, we assume that (i) there are two BPs with capacities 1500 and 5000 kg respectively, and (ii) the processing time of each of the five job-families are 13, 9, 8, 7, and 10 h, respectively. Further, in the experimental design, it is assumed that the size of the jobs varies and it is uniformly distributed between [100 kg, 1000 kg]. The uniform distribution was chosen because it is a relatively high-variance distribution, which would allow the heuristics to be tested under conditions relatively unfavorable to them (Chandru *et al.*, 1993).

The experimental design for generating test problems using the three important parameters considered in this study is summarized in Table 1

Table 1. Summary of the experimental design

Sl. no.	Factor	Levels (L)	No. of levels
1	Number of jobs (n)	L1—861; L2—943; L3—1003; L4—1107; and L5—1260	5
2	Job-priority (P)	L1—Equal probability for each job-priority; and L2—Un-equal probability for each job-priority	2
3	Job-family (F)	L1—Equal probability for each job-family; and L2—Un-equal probability for each job-family	2
Number of problem configurations			$5 \times 2 \times 2 = 20$
Problem instance per configuration			15
Total problem instances			300

and implemented in C++. For each combination of values for $\{n, P, \text{ and } F\}$, we randomly generated 15 problem instances, yielding a total of 300 ($= 5 \times 2 \times 2 \times 15$) problem instances. As a sample, the job-details of a problem instance when $n = 861$ with level 1 of $\{P \text{ and } F\}$ are given in the Appendix 1 (condensed form).

4.3. Evaluation of heuristic solutions vs. estimated optimal value

There are many procedures available in the literature for estimating optimal value for combinatorial optimization problems. We have used the procedure discussed in Rardin and Uzsoy (2001) for the statistical estimation (based on weibull distribution) of optimal value. Accordingly, for getting an estimated optimal solution for each problem instance, we need to generate a number of feasible solutions. To achieve this, a procedure, called RSA (Random Search Algorithm), is developed and the systematic procedure of this algorithm is as follows:

Step 1: = Randomly select a BP when more than one BP is available at time t , otherwise select the BP which is going to be available early for the next scheduling.

Step 2: = Randomly select a job-family from a set of feasible job-families. The criterion of feasibility may be a threshold like “above 75%” of the selected BP’s capacity, when we add the entire jobs in the selected job-family.

Step 3: = As given in Heuristic algorithm 1 (A1) for the decision: batch construction [BC].

The RSA was coded in Turbo C++. For each problem instance of the randomly generated 300 problem instances, 15 feasible solutions (that is, the local-maximum average utilization of batch processors (AUBP)) were obtained using RSA. These 15 feasible solutions were used to estimate the optimal value (estimated global-maximum AUBP) using the procedure highlighted in Rardin and Uzsoy (2001). It is to be noted that 15 feasible solutions are enough to provide the estimated optimal value of the problem instance with a very high probability of approximately 0.9999996941 $[= (1 - e^{-15})]$. Further, for each variant of the heuristic and for each problem instance belonging to the level of $(n, P, \text{ and } F)$, the heuristic-solution, the maximum AUBP, is obtained.

Results: The relative deviation (RD) of each heuristic’s solution was computed with respect to the estimated optimal solution obtained for each problem instance. For each level of $(n, P, \text{ and } F)$ the value of ARPD and MRPD were computed. Further, for each n with all levels of $(P \text{ and } F)$ the average of $\{ARPD\}$ and maximum of $\{MRPD\}$ were computed and these are shown in Figs 1 and 2, respectively.

In Fig. 1 (average performance of the heuristics), a negative ARPD indicates that on an average, the corresponding heuristic algorithm yielded a better result than the estimated optimal value and this negative ARPD varies from -0.043 to -0.195 . From this, it is possible to conclude that the heuristic algorithms (A3) and (A4), on the average yielded better utilization than the estimated optimal value. The same inference can be drawn using the worst-case performance also (see Fig. 2). This highlights that the

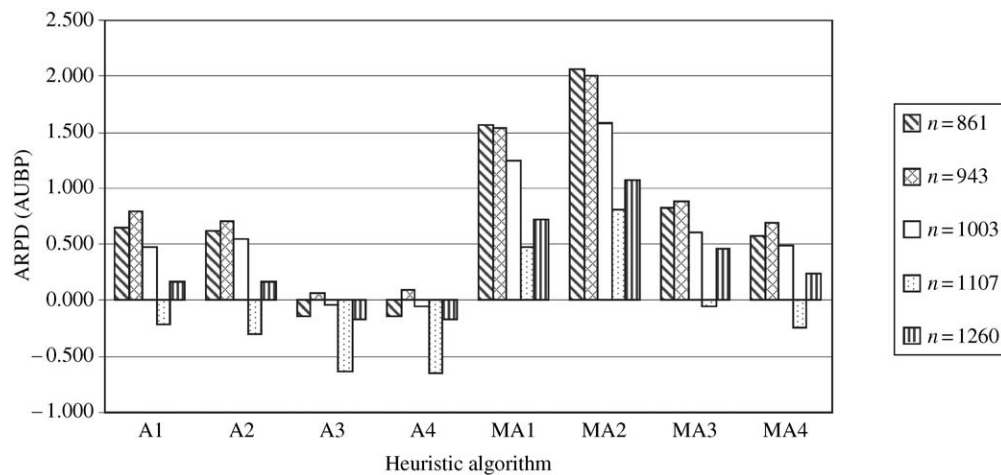


Fig. 1. Average performance of heuristics (average {ARPD w.r.t. estimated optimal AUBP}).

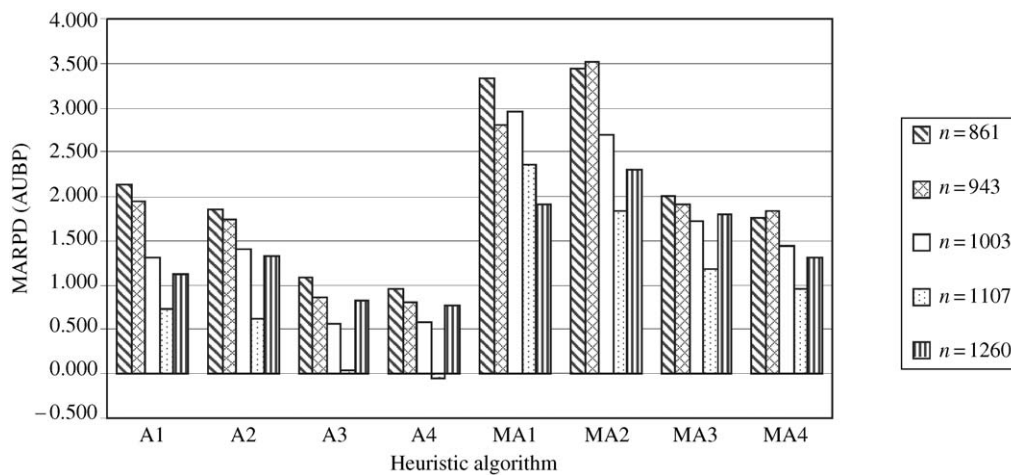


Fig. 2. Worst-case performance of heuristics (maximum {MRPD w.r.t. estimated optimal AUBP}).

weibull-based technique may have yielded conservative estimates of the optimal value. Further, in Fig. 1, a positive ARPD indicates that the heuristic algorithm resulted inferior solutions when compared with the estimated optimal value. But the deviation is very small. The same inference can be drawn using the worst-case performance also (see Fig. 2).

From Figs. 1 and 2, it can be observed that, on the average, the difference between the solution obtained from the heuristic algorithms and the estimated optimal value is not very significant, and therefore we may infer that on the average all the proposed heuristic algorithms are capable of yielding AUBP, close to (estimated) optimal AUBP.

4.4 Relative evaluation of heuristics algorithms

In this section, rather than using estimated optimal value for each problem instance (as did in the analysis of the earlier section), the relative performance of the proposed heuristics is carried out using the best (standard) solution amongst all the heuristically constructed ones and the analysis is discussed in terms of computational effort and the quality of the solutions found.

Computational effort: The computational burden of the heuristics was observed and it is low enough on average (approximately 10 CPU seconds per instance on a Pentium 200 MHz processor) for the solution of

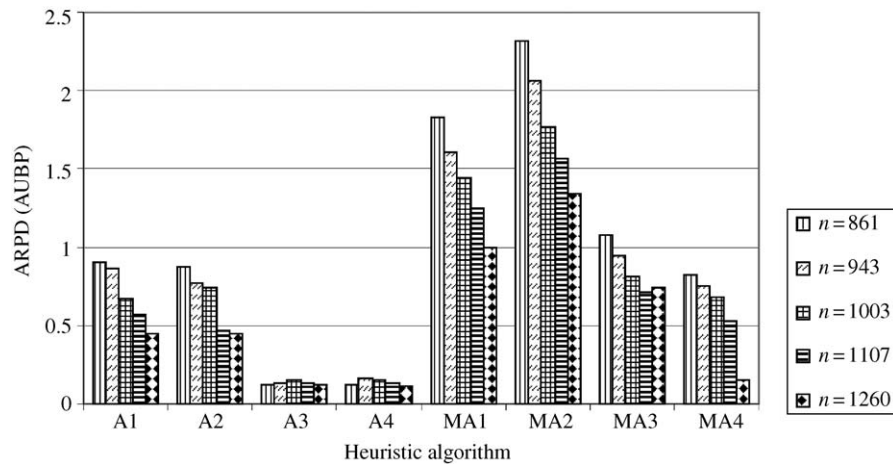


Fig. 3. Average performance of heuristics (ARPD w.r.t. primary scheduling objective—max AUBP).

large-scale problems considered in the study. Therefore, from the point of view of computational effort, we can conclude that one could run several heuristics, proposed in this paper, on each problem instance and could take the best solution found. Further, it is obvious from the logic of the algorithms that the heuristic algorithms {MA1, MA2, MA3 and MA4} will take relatively lower computational times when compared with the heuristic algorithms {A1, A2, A3, and A4}.

Quality of the solution: For each problem instance and for the primary scheduling objective of maximizing the average utilization of the batch processors, the value of the relative deviation (RD) of each heuristic algorithm were computed with respect to the best (standard) solution amongst all the heuristically constructed ones obtained for the same problem instance. For each level of (n , P , and F) the value of ARPD were computed. Further, for each n with all levels of (P and F) the average of {ARPD} is computed and these average of {ARPD} for the primary scheduling objective: maximizing AUBP is shown in Fig. 3.

For the primary scheduling objective: maximum AUBP obtained for each problem instances, the corresponding minimum weighted average waiting time (WAWT) and minimum overall flow time (OFT) were computed. In this paper these WAWT and OFT are used as secondary scheduling objectives for an effective comparison of the proposed heuristics. Further, similar computation of ARPD and average of {ARPD} were done based on the secondary

scheduling objectives. These results are shown in Figs 4 and 5 respectively.

From the Figs 3–5, there can be interesting trade-offs between the scheduling objectives of the heuristics. In order to explain this, we computed the average of AUBP, OFT, and WAWT over the 60 problem instances corresponding to each level of n . For each algorithm, the computed average value of the three scheduling objectives is given in Table 2. On the average, if we observe the quality of the solution for the primary scheduling objective-maximizing AUBP, the difference between the best solution and the worst solution is insignificant (see Table 2). However, for the secondary scheduling objectives, viz., minimizing OFT and WAWT, the differences are very high (see Table 2). Hence, it may be useful to consider heuristics that are inferior with respect to AUBP (because the difference between the best solution and the worst solution is insignificant) but superior with respect to both OFT and WAWT.

In particular, it is evident from Table 2 that the second set of heuristics {MA1, MA2, MA3, and MA4} should be preferred to the first set of heuristics {A1, A2, A3, and A4}. That is, the performance of the second set of heuristics with respect to secondary scheduling objectives (that is minimizing OFT and minimizing WAWT) is superior in comparison with first set of heuristics. Further, with respect to the primary scheduling objective (that is maximizing AUBP) the second set of heuristics is almost comparable with the first set of heuristics. Continuing with this line of reasoning, if we were to select a single heuristic, it is clear that

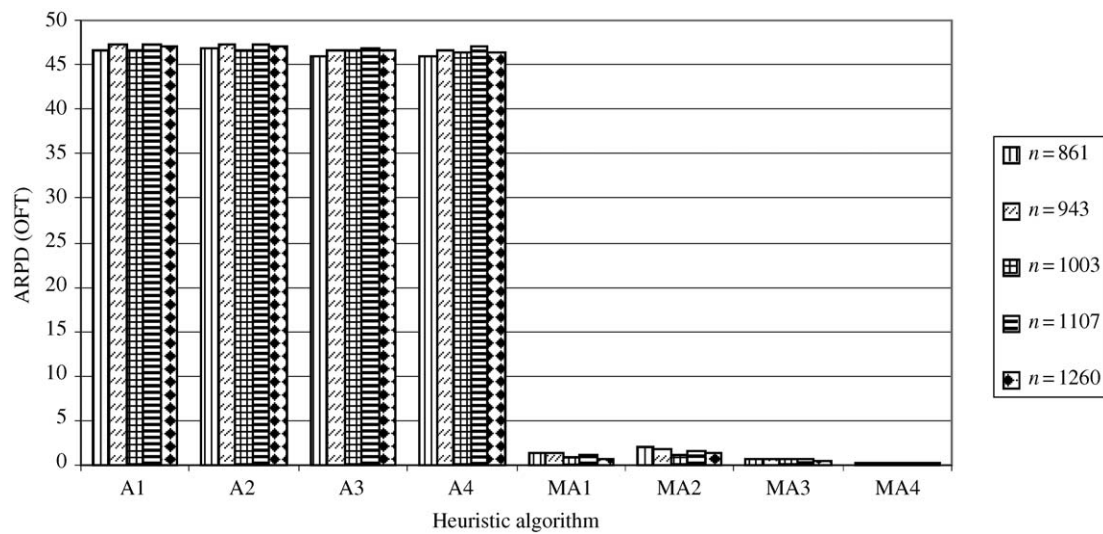


Fig. 4. Average performance of heuristics (ARPD w.r.t. secondary scheduling objective—min. OFT).

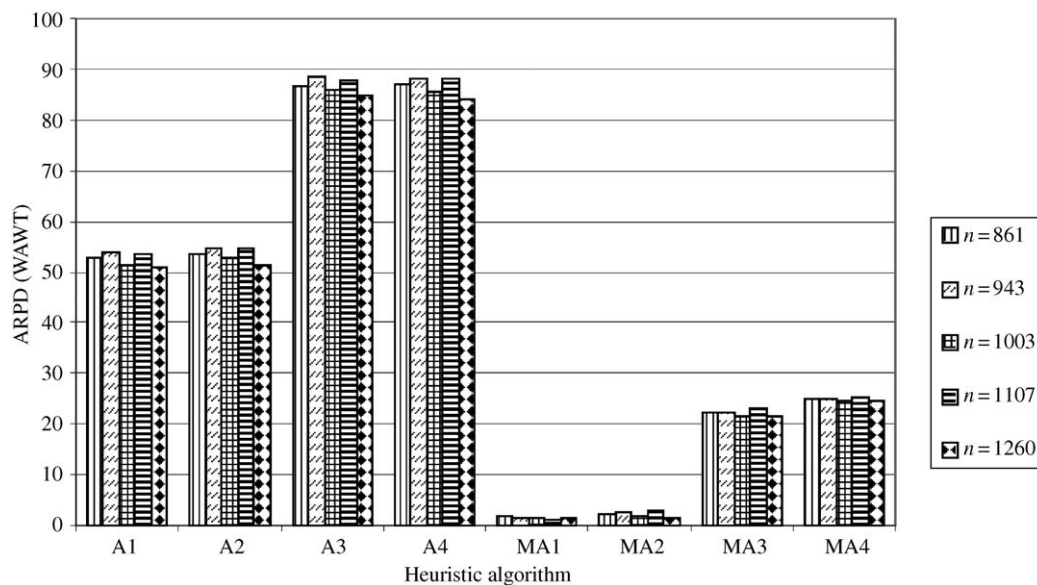


Fig. 5. Average performance of heuristics (ARPD w.r.t. secondary scheduling objective—min. WAWT).

the trade-offs between AUBP, OFT and WAWT will be in favor of heuristic MA1. The same inference was hold good for the results obtained based on the worst-case performance analysis using the value of MRPD.

From the detailed results of ARPD (not provided here for the brevity of the paper) for each level of (n , P and F), it is clear that there is an influence of problem

instance on the performance of the heuristic algorithms for all the scheduling objectives. This inference was further verified statistically based on the non-parametric ANOVA (non-parametric ANOVA was used because the data failed in normality-test) using WINDOW based statistical package SIGMASTAT. Further, it shows that there is variability in the performance of the heuristics with the problem factors

Table 2. Average performance of heuristics vs. scheduling objectives

No. of jobs (<i>n</i>)	Scheduling objectives	Average performance of the heuristic algorithm								Best solution— worst solution
		<i>A1</i>	<i>A2</i>	<i>A3</i>	<i>A4</i>	<i>MA1</i>	<i>MA2</i>	<i>MA3</i>	<i>MA4</i>	
861	SO-1	95.88	95.91	96.63	96.64	94.99	94.51	95.71	95.95	2.13
	SO-2	1064	1066	1059	1060	737	741	731	727	339
	SO-3	89	90	109	109	60	60	72	73	49
943	SO-1	96.04	96.12	96.74	96.72	95.31	94.87	95.96	96.14	1.87
	SO-2	1169	1169	1163	1163	804	808	799	796	373
	SO-3	100	101	123	122	66	67	80	81	57
1003	SO-1	96.35	96.28	96.85	96.86	95.60	95.28	96.22	96.34	1.58
	SO-2	1240	1241	1238	1236	854	857	849	846	395
	SO-3	107	108	132	131	72	72	86	88	60
1107	SO-1	96.60	96.69	97.02	97.02	95.93	95.62	96.45	96.63	1.40
	SO-2	1367	1368	1363	1364	939	943	934	930	438
	SO-3	120	120	146	147	80	80	97	98	67
1260	SO-1	96.89	96.89	97.22	97.22	96.35	96.02	96.61	96.83	1.20
	SO-2	1547	1547	1542	1540	1060	1066	1058	1054	493
	SO-3	138	138	168	167	93	93	111	113	75

SO-1: Avg. {AUBP} in percentage; SO-2: Avg. {OFT} in hours; SO-3: Avg. {WAWT} in hours.

(*n*, *P* and *F*). That is, there is an influence of the problem factors on the performance of heuristic algorithms.

5. Effect of family-processing time

In this part of the analysis, we study the influence of changes in the processing time of job-families on the performance of the heuristic algorithms. The design of the computational experiments and the results are detailed in the following sections.

5.1. Computational experiments

The processing time of five job-families (13, 9, 8, 7, and 10 h, respectively) used in the experimental design detailed in Section 4.2 were re-used here to generate systematically various sets of processing times for the five job-families. In this study, the processing time of each family is re-assigned systematically among the five job-families in a maximum of 5! (= 120) ways. These 120 family-processing time sets were generated and the same is given in Appendix 2. Further, from the 300 problem instances used in the earlier computational experiments, 20 problem instances, which are representing one problem instance for each level of {*n*, *P* and *F*}, are re-used

to study the effect of change in family processing time on the performance of the algorithms.

In this experiment, each problem instance is replicated 120 times and in each run one job-family processing time set is used from the 120 distinct job-family processing time set (as given in Appendix 2). Thus we have 2400 runs for each of the eight heuristics. The results obtained from the 2400 are presented in the following section.

5.2 Results and discussions

Tables 3 and 4 detail the average and worst-case performance of the heuristics with respect to the scheduling objective-maximizing AUBP. For each problem instance, the best algorithm and the worst algorithm among the eight variants after replicating 120 runs, is provided in the last column of the Tables 3 and 4 respectively. For each level of *n*, the average performance and worst performance of the algorithms were computed and reported in the Tables 3 and 4 respectively and appropriately. Further, a grand weighted average performance and worst performance of the algorithms were computed using the average scores corresponding to each level of *n* and the same is provided in the last row of these two tables respectively.

From Tables 3 and 4, it is clear that for each

Table 3. Effect of changes in job-family processing time on heuristic performance—ARPD (AUBP)

No. of runs	Problem instance	Average performance of heuristic algorithms—ARPD (AUBP)								Best algorithm
		A1	A2	A3	A4	MA1	MA2	MA3	MA4	
120	1 ($n=L1, P=L1, F=L1$)	1.02	1.60	1.34	1.02	0.85	1.44	1.14	0.72	MA4
120	2 ($n=L1, P=L1, F=L2$)	1.95	1.75	0.54	0.57	1.32	0.34	2.03	1.42	MA2
120	3 ($n=L1, P=L2, F=L1$)	1.03	1.19	0.60	0.78	1.03	1.22	0.85	0.61	A3
120	4 ($n=L1, P=L2, F=L2$)	1.19	1.07	0.23	0.46	1.07	0.48	1.90	1.91	A3
	Average	1.30	1.40	0.68	0.71	1.06	0.87	1.48	1.17	A3
120	5 ($n=L2, P=L1, F=L1$)	1.53	1.59	1.39	1.18	0.98	2.00	0.49	0.37	MA3
120	6 ($n=L2, P=L1, F=L2$)	1.34	1.53	0.37	0.29	0.66	0.48	1.52	1.37	A4
120	7 ($n=L2, P=L2, F=L1$)	1.33	1.77	1.50	0.98	1.09	1.71	1.07	0.16	MA4
120	8 ($n=L2, P=L2, F=L2$)	1.63	2.02	0.39	0.34	0.90	0.43	2.19	2.35	A4
	Average	1.46	1.73	0.91	0.70	0.90	1.16	1.32	1.06	A4
120	9 ($n=L3, P=L1, F=L1$)	1.44	1.49	1.45	1.24	0.79	1.27	0.98	0.14	MA4
120	10 ($n=L3, P=L1, F=L2$)	1.22	1.11	0.59	0.55	0.74	0.23	1.38	1.89	MA2
120	11 ($n=L3, P=L2, F=L1$)	1.22	1.33	1.20	0.85	0.96	0.69	0.70	0.57	MA4
120	12 ($n=L3, P=L2, F=L2$)	1.45	1.23	0.28	0.46	0.78	0.33	1.41	1.10	A3
	Average	1.33	1.29	0.88	0.78	0.82	0.63	1.12	0.93	MA2
120	13 ($n=L4, P=L1, F=L1$)	0.84	1.23	0.55	0.49	0.60	1.07	0.78	0.27	MA4
120	14 ($n=L4, P=L1, F=L2$)	1.17	1.39	0.43	0.43	1.19	0.18	1.76	1.16	MA2
120	15 ($n=L4, P=L2, F=L1$)	1.27	1.59	1.02	0.95	0.52	0.88	0.76	0.27	MA4
120	16 ($n=L4, P=L2, F=L2$)	0.69	1.01	0.27	0.28	0.68	0.19	1.18	0.98	MA3
	Average	0.99	1.31	0.57	0.54	0.75	0.58	1.12	0.67	A4
120	17 ($n=L5, P=L1, F=L1$)	0.51	0.74	0.66	0.50	0.33	0.63	0.52	0.36	MA1
120	18 ($n=L5, P=L1, F=L2$)	1.09	0.88	0.46	0.42	0.90	0.53	0.66	0.56	A4
120	19 ($n=L5, P=L2, F=L1$)	0.82	1.09	0.71	0.46	0.32	0.55	0.74	0.32	MA1, MA4
120	20 ($n=L5, P=L2, F=L2$)	1.31	1.12	0.22	0.30	0.58	0.41	1.04	0.88	A3
	Average	0.93	0.96	0.51	0.42	0.53	0.53	0.74	0.53	A4
	Grand Average	1.18	1.31	0.70	0.61	0.79	1.12	0.73	0.84	A4

Each cell value indicates the average performance of the heuristics over 120 run (i.e., 120 “Job-family processing time” set).

algorithm, the values of ARPD and MRPD significantly vary when the processing time of job-families changes. Further, the choice of the best algorithm and worst algorithm based on ARPD and MRPD respectively varies with respect to the scheduling objective of maximizing AUBP. This indicates that the processing time set of job-families influences the performance of the heuristic algorithms. Further, this inference was statistically verified using a two-way ANOVA (Factors: job-family processing time sets (120 Levels), heuristic algorithms (eight levels); Dependent Variables: ARPD, MRPD) using the Package SIGMASTAT. Since the normality test for the specific data failed and the equal variance was valid, a non-parametric ANOVA was carried out. From the results obtained the influence was found to be statistically significant.

6. Conclusion

We have addressed the more general case of scheduling heterogeneous batch processors with incompatible job families and non-identical job sizes, which occurs in many situations, including steel casting manufacturing, semiconductor manufacturing and metal manufacturing industries. The scheduling problem has three interrelated decisions. First, we have to decide a BP when ever more than one heterogeneous batch processors available simultaneously for loading jobs, second we have to decide a job-family from among the available incompatible job-families for selecting a set of jobs for the batch processor decided in the first stage, and lastly we have to decide a set of jobs from the selected job-family for loading into the selected batch processor.

We highlighted the complexity of the scheduling

Table 4. Effect of changes in job-family processing time on heuristic performance—MRPD (AUBP)

No. of runs	Problem instance (No. of jobs)	Worst case performance of heuristic algorithms—MRPD (AUBP)								Worst algorithm
		A1	A2	A3	A4	MA1	MA2	MA3	MA4	
120	1 ($n=L1, P=L1, F=L1$)	2.90	3.82	3.12	3.28	2.64	2.46	3.06	3.67	A2
120	2 ($n=L1, P=L1, F=L2$)	5.26	4.58	1.65	2.11	3.99	4.57	3.33	1.37	A1
120	3 ($n=L1, P=L2, F=L1$)	3.49	3.13	2.84	2.31	2.91	2.45	1.98	3.21	A1
120	4 ($n=L1, P=L2, F=L2$)	3.79	3.78	1.22	1.82	3.56	3.82	3.39	1.87	MA2
	Average	3.86	3.83	2.21	2.38	3.28	3.33	2.94	2.53	A1
120	5 ($n=L2, P=L1, F=L1$)	3.12	3.47	3.47	3.01	2.83	2.12	2.06	4.13	MA4
120	6 ($n=L2, P=L1, F=L2$)	3.22	3.59	1.23	1.33	2.59	3.15	3.65	1.56	MA3
120	7 ($n=L2, P=L2, F=L1$)	2.82	3.40	3.79	3.50	2.88	2.79	1.08	3.46	A3
120	8 ($n=L2, P=L2, F=L2$)	4.27	4.74	1.34	1.72	3.26	3.76	4.18	1.94	A2
	Average	3.36	3.80	2.46	2.39	2.89	2.96	2.74	2.77	A2
120	9 ($n=L3, P=L1, F=L1$)	3.43	3.64	3.63	3.99	2.53	2.53	0.98	3.34	A4
120	10 ($n=L3, P=L1, F=L2$)	3.44	2.36	1.45	1.80	2.86	2.69	3.27	1.06	A1
120	11 ($n=L3, P=L2, F=L1$)	2.62	3.13	3.03	2.41	2.63	2.58	1.86	1.97	A2
120	12 ($n=L3, P=L2, F=L2$)	2.80	2.95	1.71	1.73	2.58	3.11	2.64	1.35	MA2
	Average	3.07	3.02	2.46	2.48	2.65	2.73	2.19	1.93	A1
120	13 ($n=L4, P=L1, F=L1$)	2.14	3.72	2.01	1.84	2.47	1.72	1.49	2.45	A2
120	14 ($n=L4, P=L1, F=L2$)	3.31	3.03	1.27	1.53	3.23	3.28	3.14	1.06	A1
120	15 ($n=L4, P=L2, F=L1$)	3.18	3.23	2.87	3.51	2.70	2.01	1.56	2.45	A2
120	16 ($n=L4, P=L2, F=L2$)	1.88	2.80	1.14	1.01	2.38	2.43	2.22	0.92	A2
	Average	2.63	3.20	1.82	1.97	2.70	2.36	2.10	1.72	A2
120	17 ($n=L5, P=L1, F=L1$)	1.43	1.52	2.27	1.67	1.29	1.48	1.58	1.89	A3
120	18 ($n=L5, P=L1, F=L2$)	2.29	2.89	0.98	1.20	3.43	2.75	1.79	1.47	MA1
120	19 ($n=L5, P=L2, F=L1$)	2.13	2.63	2.29	1.98	1.18	2.06	1.28	2.10	A2
120	20 ($n=L5, P=L2, F=L2$)	1.43	1.52	2.27	1.67	1.29	1.48	1.58	1.89	A3
	Average	1.82	2.14	1.95	1.63	1.80	1.94	1.56	1.84	A2
	Grand Average	2.86	3.12	2.16	2.13	2.60	2.60	2.24	2.12	A2

Each cell value indicates the worst performance of the heuristics over 120 run (i.e., 120 Job-family processing time set).

problem discussed in this paper and proposed greedy heuristics, which are mostly related to managerial considerations in the scheduling of heat treatment operations in steel casting manufacturing. The empirical analyzes of the heuristics that have been described in this paper for the research problem leads to the following main inferences:

- The sub-optimality of the heuristics with respect to the statistically estimated optima (based on the Weibull technique) was shown to be uniformly small. Further, on the average the heuristics resulted the maximum average utilization within the range of 95% to 97.2% (Table 2). This suggests that all the eight variants of the greedy heuristics are acceptable based on the primary scheduling objective of maximizing the average utilization of batch processors.
- From the point of view of computational effort and from the yield on average utilization of batch processors, we can conclude that one could run several heuristics, proposed in this paper on each problem instance and could take one that gives the best solution.
- In general, the performance of the heuristic was seen to be sensitive to the problem factor (n , P , and F).
- The three algorithms A3, MA4 and MA1 were found to be superior with respect to the primary scheduling objective of maximizing average utilization of batch processors (AUBP), the secondary scheduling objectives of minimization of overall flow time (OFT) and minimizing weighted average waiting time (WAWT) respectively. Further, the heuristic algorithm MA1

turns out to be the best choice if we trade-off all three objectives AUBP, OFT and WAWT, based on the data used in this paper.

From the problem instances considered and computational experiments carried out for the simple sensitivity analysis, it appears in general that there is an influence of changes in the input parameters “processing time set of job-families” on the performance of the proposed heuristic algorithms. However, on the average performance and on the worst-case performance of the heuristics, the difference among the values of ARPD and MRPD respectively, is minimal. Therefore, on the average, the choice of best algorithm for maximizing average utilization of batch processors almost remain unchanged.

There are a number of interesting extensions of the problems can be pursued. One such consideration is how the performance of the proposed heuristic algorithms is affected when we allow the changes individually as well as simultaneously in the input parameters such as (a) job-size distribution and (b) a specific capacity combination of the two non-identical batch processors. Further, the most obvious limitation of the proposed heuristics is the lack of due dates. Thus, another important extension could be to include in the proposed model is the due date related performance measures.

Acknowledgments

The authors would like to acknowledge the referees for their helpful comments and suggestions. This work was partly supported by the Singapore-MIT Alliance, School of Mechanical and Production Engineering, Nanyang Technological University, Singapore. This support is greatly acknowledged.

Notes

1. This parameter is given as a variable in the computer implementation of the proposed greedy heuristics and thus the algorithms accepts new jobs dynamically in any fixed interval of time.
2. In the observed steel-casting foundry, priority of a job is assigned based on three criteria (1) value, (2) alloy type and (3) market status of the order with two alternatives for each criterion (that is, high value vs. low value; high alloy vs. low alloy; and export order vs. domestic order). Thus, in this paper the priority of a job varies from 1 to 8.

Appendix 1. A sample problem instance [$n = 861$ Jobs and level 1 for both P and F]

Day	Job code	Job size	Job priority	Job family
1	1	252	3	1
1	2	351	1	3
1	3	409	8	1
1	4	855	6	5
.	.	.	.	.
1	117	559	3	3
1	121	886	1	1
1	122	904	7	5
1	123	955	2	2
2	1	914	4	1
2	2	312	7	4
2	3	564	2	4
2	4	325	5	2
.	.	.	.	.
2	120	649	7	4
2	121	692	8	1
2	122	263	3	2
2	123	738	1	5
3	1	114	7	2
3	2	974	5	3
3	3	584	2	4
3	4	726	3	2
.	.	.	.	.
3	120	592	5	1
3	121	388	1	2
3	122	454	6	3
3	123	834	1	1
4	1	212	3	3
4	2	721	8	5
4	3	660	6	2
4	4	890	1	2
.	.	.	.	.
4	120	786	8	4
4	121	531	2	5
4	122	410	7	4
5	123	284	3	4
5	1	194	5	4
5	2	614	6	5
5	3	668	5	5
5	4	506	2	1
.	.	.	.	.
5	120	601	7	5
5	121	861	7	5
5	122	226	3	1
5	123	416	7	1
6	1	934	8	5
6	2	760	5	1
6	3	432	1	5
6	4	354	7	1

Appendix 1. (Continued)

<i>Day</i>	<i>Job code</i>	<i>Job size</i>	<i>Job priority</i>	<i>Job family</i>
.	.	.	.	.
6	120	596	7	5
6	121	828	7	1
6	122	220	7	3
6	123	227	2	5
7	1	742	1	2
7	2	832	1	5
7	3	891	4	2
7	4	584	5	5
.	.	.	.	.
7	119	654	8	2
7	120	114	4	5
7	121	605	3	2
7	122	201	8	1
7	123	371	7	3

Appendix 2. Generated processing time sets of the job-families (120 sets)

<i>Run (set)</i>	<i>Processing time of job-families</i>				
	<i>F1</i>	<i>F2</i>	<i>F3</i>	<i>F4</i>	<i>F5</i>
1	13	9	8	7	10
2	13	9	8	10	7
3	13	9	7	8	10
4	13	9	7	10	8
5	13	9	10	8	7
6	13	9	10	7	8
7	13	8	9	7	10
8	13	8	9	10	7
9	13	8	7	9	10
10	13	8	7	10	9
11	13	8	10	9	7
12	13	8	10	7	9
13	13	7	9	8	10
14	13	7	9	10	8
15	13	7	8	9	10
16	13	7	8	10	9
17	13	7	10	9	8
18	13	7	10	8	9
19	13	10	9	8	7
20	13	10	9	7	8
21	13	10	8	9	7
22	13	10	8	7	9
23	13	10	7	9	8
24	13	10	7	8	9
25	9	13	8	7	10
26	9	13	8	10	7

Appendix 2. (Continued)

<i>Run (set)</i>	<i>Processing time of job-families</i>				
	<i>F1</i>	<i>F2</i>	<i>F3</i>	<i>F4</i>	<i>F5</i>
27	9	13	7	8	10
28	9	13	7	10	8
29	9	13	10	8	7
30	9	13	10	7	8
31	9	8	13	7	10
32	9	8	13	10	7
33	9	8	7	13	10
34	9	8	7	10	13
35	9	8	10	13	7
36	9	8	10	7	13
37	9	7	13	8	10
38	9	7	13	10	8
39	9	7	8	13	10
40	9	7	8	10	13
41	9	7	10	13	8
42	9	7	10	8	13
43	9	10	13	8	7
44	9	10	13	7	8
45	9	10	8	13	7
46	9	10	8	7	13
47	9	10	7	13	8
48	9	10	7	8	13
49	8	13	9	7	10
50	8	13	9	10	7
51	8	13	7	9	10
52	8	13	7	10	9
53	8	13	10	9	7
54	8	13	10	7	9
55	8	9	13	7	10
56	8	9	13	10	7
57	8	9	7	13	10
58	8	9	7	10	13
59	8	9	10	13	7
60	8	9	10	7	13
61	8	7	13	9	10
62	8	7	13	10	9
63	8	7	9	13	10
64	8	7	9	10	13
65	8	7	10	13	9
66	8	7	10	9	13
67	8	10	13	9	7
68	8	10	13	7	9
69	8	10	9	13	7
70	8	10	9	7	13
71	8	10	7	13	9
72	8	10	7	9	13
73	7	13	9	8	10
74	7	13	9	10	8
75	7	13	8	9	10

Appendix 2. (Continued)

Run (set)	Processing time of job-families				
	F1	F2	F3	F4	F5
76	7	13	8	10	9
77	7	13	10	9	8
78	7	13	10	8	9
79	7	9	13	8	10
80	7	9	13	10	8
81	7	9	8	13	10
82	7	9	8	10	13
83	7	9	10	13	8
84	7	9	10	8	13
85	7	8	13	9	10
86	7	8	13	10	9
87	7	8	9	13	10
88	7	8	9	10	13
89	7	8	10	13	9
90	7	8	10	9	13
91	7	10	13	9	8
92	7	10	13	8	9
93	7	10	9	13	8
94	7	10	9	8	13
95	7	10	8	13	9
96	7	10	8	9	13
97	10	13	9	8	7
98	10	13	9	7	8
99	10	13	8	9	7
100	10	13	8	7	9
101	10	13	7	9	8
102	10	13	7	8	9
103	10	9	13	8	7
104	10	9	13	7	8
105	10	9	8	13	7
106	10	9	8	7	13
107	10	9	7	13	8
108	10	9	7	8	13
109	10	8	13	9	7
110	10	8	13	7	9
111	10	8	9	13	7
112	10	8	9	7	13
113	10	8	7	13	9
114	10	8	7	9	13
115	10	7	13	9	8
116	10	7	13	8	9
117	10	7	9	13	8
118	10	7	9	8	13
119	10	7	8	13	9
120	10	7	8	9	13

References

- Azizoglu, M. and Webster, S. (2001) Scheduling a batch processing machine with incompatible job families. *Computers and Industrial Engineering*, **39**, 325–335.
- Azizoglu, M. and Webster, S. (2000) Scheduling a batch processing machine with non-identical job-sizes. *International Journal of Production Research*, **38**(10), 2173–2184.
- Chambers, R. J., Carraway, R. L., Lowe, T. J. and Morin, T. L. (1991) Dominance and decomposition heuristics for single machine scheduling. *Operations Research*, **39**, 639–647.
- Chandru, V., Lee, C.-Y. and Uzsoy, R. (1993) Minimizing total completion time on a batch-processing machine. *International Journal of Production Research*, **31**(9), 2097–2121.
- Dobson, G. and Nambimadom, R. S. (1994) The batch loading and scheduling problem. Working Paper No. QM 92-03, Simon School of Business Administration, University of Rochester, Rochester, N.Y., U.S.A.
- Dobson, G. and Nambimadom, R. S. (2001) The batch loading and scheduling problem. *Operations Research*, **49**(1), 52–65.
- Devpura, A., Fowler, J. W., Carlyle, M. W. and Perez, I. (2000) Minimizing total weighted tardiness on single batch process machine with incompatible job families. <http://www.uni-duisburg.de/FB5/BWL/WI/or2000/sektion10/fowler.pdf>.
- Fanti, M. P., Maione, B., Piscitelli, G. and Turchiano, B. (1996) Heuristic scheduling of jobs on a multi-product batch-processing machine. *International Journal of Production Research*, **34**(8), 2163–2186.
- Hung, Yi-Feng. (1998) Scheduling of mask shop E-beam writers. *IEEE Transactions on Semiconductor Manufacturing*, **11**(1), 165–172.
- Kempf, K. G., Uzsoy, R. and Wang C. S. (1998) Scheduling a single batch processing machine with secondary resource constraints. *Journal of Manufacturing Systems*, **17**(1), 37–51.
- Kim, H.-U., Kim, Y.-D. and Kim, J.-G. (2000) Batching and scheduling at a batch processing workstation in a semiconductor fabrication facility producing multiple product types with distinct due dates. *Proceedings of the International Conference on Modeling and Analysis of Semiconductor Manufacturing*, pp. 151–156.
- Mathirajan, M., Sivakumar, A. I. (2003) Scheduling batch processors in semiconductor manufacturing—A review. *Proceedings of Singapore MIT Alliance (SMA)—2003 Symposium*, National University of Singapore, Singapore, <https://dspace.mit.edu/handle/1721.1/3755>.

- Mehta, S.V. and Uzsoy, R. (1998) Minimizing total tardiness on a batch-processing machine with incompatible job families. *IIE Transactions*, **30**, 165–178.
- Neuts, M. F. (1967) A general class of bulk queues with Poisson input. *Annals of Mathematical Statistics*, **38**, 759–770.
- Rardin, R. L. and Uzsoy, R. (2001). Experimental evaluation of heuristic optimization algorithms: A tutorial. *Journal of Heuristics*, **7**(3), 261–304.
- Shekar, G. L. (1998) *Planning and scheduling systems for steel casting production—A new paradigm*, Ph.D. Dissertation, Department of Management Studies, Indian Institute of Science, Bangalore, India.
- Uzsoy, R. (1995) Scheduling batch processing machine with incompatible job-families. *International Journal of Production Research*, **33**(10), 2685–2708.
- Uzsoy, R., Lee, C.-Y. and Martin-Vega, L. A. (1992) Scheduling semiconductor test operations: minimizing maximum lateness and number of tardy jobs on a single machine. *Naval Research Logistics*, **39**, 369–388.
- Vepsäläinen, A. and Morton, T. E. (1987) Priority rules and lead-time estimation for job-shop scheduling with weighted tardiness costs. *Management Science*, **33**(8), 1035–1047.
- Zee, D. J., Van Harten, A. and Schuur, P. C. (1997) Dynamic job assignment heuristics for multi-server batch operations—a cost based approach. *International Journal of Production Research*, **35**(11), 3063–3093.
- Zee, D. J. van der, Van Harten, A. and Schuur, P. C. (2001) On-line scheduling of multi-server batch operations, *IIE Transactions*, **33**, 569–586.